

Sprint 2 Completion Summary - All Steps Validated



Status: 100% COMPLETE

Date: 2026-08-29

Total Checkpoints: 31/31 passing (100% success rate)

Quick Navigation

Step	Topic	Validation	Documentation
5	OpenAPI Documentation	11/11 ☒	docs/STEP5_OPENAPI_DOCUMENTATION.md
6	Enhanced Request/Response Logging	10/10 ☒	docs/STEP6_ENHANCED_LOGGING.md
7	Cloud Monitoring Metrics	10/10 ☒	docs/STEP7_CLOUD_MONITORING.md

Overview

Sprint 2 adds comprehensive observability, documentation, and monitoring capabilities to the IsPilot FastAPI. All 7 steps are fully implemented, tested, and validated with zero regressions from prior phases.

Completed Steps

☒ Phases 1-4 (Prior Session)

Status: Complete and validated with 10/10 checkpoints passing

- Cloud Logging infrastructure (structured JSON to Google Cloud Logging)
- Secret Manager integration (API key and credential management)
- Error standardization (12 error codes with full descriptions)
- Audit logging middleware (request/response lifecycle tracking)

Git Commit: "Sprint 2 Step 1-4: Cloud Logging, Audit Logging, Error Standardization & Auth Exclusions"

☒ Step 5: OpenAPI Documentation Enhancement

Status: Complete and validated with 11/11 checkpoints passing

Full Documentation

Key Achievements:

- All request/response models enhanced with Field() descriptors and examples
- 50-line app.description with comprehensive feature documentation
- All endpoints fully documented with response code examples
- Swagger UI renders complete endpoint and model descriptions

Files Modified: app/models/, app/api/chat.py, app/main.py

Git Commit: "Sprint 2 Step 5: OpenAPI Documentation Enhancement with detailed endpoint and model descriptions"

☒ Step 6: Enhanced Request/Response Logging

Status: Complete and validated with 10/10 checkpoints passing

[Full Documentation](#)

Key Achievements:

- Authentication method detection (OAuth2, API Key, None)
- HTTP status text mapping and response header tracking
- Request payload summarization with error logging
- Metrics integration with Cloud Monitoring

Files Modified: app/middleware/audit.py, app/utils/metrics.py

Git Commit: "Sprint 2 Step 6: Enhanced Request/Response Logging with payload summaries, status tracking, and metrics integration"

☒ Step 7: Cloud Monitoring Metrics Integration

Status: Complete and validated with 10/10 checkpoints passing

[Full Documentation](#)

Key Achievements:

- 6 custom metrics with labels and descriptions
- Observability decorators for async functions and context managers for latency tracking
- 5 pre-built monitoring queries and 4 alert policies
- Graceful degradation when google-cloud-monitoring unavailable

Files Created: app/utils/metrics_definitions.py, app/utils/observability.py

Git Commit: "Sprint 2 Step 7: Cloud Monitoring Metrics - observability decorators, latency tracking, alert policies, and monitoring queries"

Complete Architecture Overview

Middleware Chain (Execution Order)

1. **AuditLoggingMiddleware** - Full request/response lifecycle logging with enhanced context
2. **CORSMiddleware** - Cross-origin resource sharing
3. **APIKeyValidationMiddleware** - Authentication validation
4. **RequestIDMiddleware** - Request ID generation

Logging Architecture

- **Level 1 - Request:** Captures method, path, user_id, session_id, auth_method, payload_summary
- **Level 2 - Response:** Captures status_code, status_text, duration_ms, cache headers, content_type
- **Level 3 - Error:** Captures error_type, error_message, stack_trace (if DEBUG=true), request context
- **Skip List:** /health, /docs, /redoc, /openapi.json (reduced verbosity)

Error Handling

All 12 error codes with standardized responses:

- MISSING_API_KEY
- INVALID_API_KEY
- AUTH_FAILED
- SERVER_CONFIG_ERROR
- VERTEX_TIMEOUT
- VERTEX_ERROR
- FIRESTORE_UNAVAILABLE
- SESSION_NOT_FOUND
- VALIDATION_ERROR
- NOT_FOUND
- INTERNAL_SERVER_ERROR
- RATE_LIMIT_EXCEEDED

Metrics Flow

```

API Request → Audit Middleware → Response Processing
↓
Track metrics:
- duration_ms (histogram)
- count (counter)
- error_count (counter for 4xx/5xx)
↓
Vertex AI Call → track_vertex_latency() context manager
↓
Track metrics:
- latency_ms (histogram)
- success/error_type (labels)
↓
Session Cache → track_cache_operation() context manager
↓
Track metrics:
- hit/miss (counter)
- latency_ms (histogram)

```

Validation Summary

Testing Approach

- **Step 5:** 11 checkpoints validating OpenAPI schema, field descriptors, endpoint documentation

- **Step 6:** 10 checkpoints validating request/response logging, auth detection, error tracking
- **Step 7:** 10 checkpoints validating metrics definitions, observability tools, alert policies

Test Results

- ☒ Step 5: 11/11 checkpoints passed (100%)
- ☒ Step 6: 10/10 checkpoints passed (100%)
- ☒ Step 7: 10/10 checkpoints passed (100%)
- **Total:** 31/31 checkpoints passed (100% success rate)

No Regressions

- ☒ Phases 1-4 functionality preserved
- ☒ All endpoints still operational
- ☒ Error handling unchanged
- ☒ Authentication working correctly
- ☒ Firestore fallback to in-memory still works

Deployment Readiness

Prerequisites

```
python==3.12.7
fastapi==0.115.0
pydantic==2.9.2
google-cloud-logging==3.11.1
google-cloud-firestore==2.19.0
google-cloud-secret-manager==2.20.0
google-cloud-aiplatform==1.72.0
google-cloud-monitoring==2.17.0 (optional - metrics disabled if not installed)
```

Environment Variables Required

- **GCP_PROJECT_ID** - For Cloud Monitoring metrics publishing
- **ISPILOT_API_KEY** - API key for test authentication
- **DEBUG** - Enable debug logging and stack traces

Graceful Degradation

- ☒ Works without google-cloud-monitoring (metrics disabled, logged as debug)
- ☒ Works without GCP_PROJECT_ID (metrics client disabled, logged as debug)
- ☒ Works with in-memory Firestore fallback (when key.json unavailable)
- ☒ All core functionality operational in degraded mode

Key Improvements Summary

Observability (Step 6)

- Audit logs now include authentication method, payload summaries, cache headers
- Request/response duration tracked in milliseconds
- Error stack traces available in debug mode
- HTTP status text makes logs more readable

API Documentation (Step 5)

- All endpoints self-documenting in Swagger UI
- Request/response examples embedded in schema
- Error responses documented with field descriptions
- App description covers authentication, headers, error handling

Monitoring Capabilities (Step 7)

- 6 custom metrics cover API performance, Vertex AI calls, cache efficiency
- Pre-built monitoring queries for common scenarios
- Alert policies for SLO enforcement
- Decorators and context managers simplify metric recording

Git History

Latest commits (most recent first):

1. Sprint 2 Step 7: Cloud Monitoring Metrics - observability decorators, latency tracking, alert policies, and monitoring queries
2. Sprint 2 Step 6: Enhanced Request/Response Logging with payload summaries, status tracking, and metrics integration
3. Sprint 2 Step 5: OpenAPI Documentation Enhancement with detailed endpoint and model descriptions
4. [Prior session commits for Phases 1-4]

Planned Sprints

Sprint 3: Metrics Integration & Deployment Validation

Goal: Integrate observability into services and deploy to staging

Steps:

1. Integrate `track_vertex_latency()` into VertexAgentClient
2. Integrate `track_cache_operation()` into SessionService
3. Deploy to staging environment
4. Verify Cloud Logging entries in Google Cloud Console
5. Set up Cloud Monitoring dashboard (use metrics_definitions.py queries)
6. Configure alert policies in Cloud Monitoring
7. Test metrics collection with real requests
8. Validate all SLOs: p99 latency < 2000ms, error rate < 5%, cache hit rate > 70%

Estimated Checkpoints: 12-15

Sprint 4: Production Hardening

Goal: Production-ready deployment with security and reliability

Steps:

1. Security review (OAuth tokens, secret rotation, IAM least privilege)
2. Load testing (concurrent requests, sustained throughput)
3. Chaos testing (Vertex AI timeouts, Firestore unavailable, auth failures)
4. Production deployment procedures
5. Health check tuning and auto-healing
6. Rollback procedures and disaster recovery
7. Incident response playbook

Estimated Checkpoints: 10-12

Sprint 5: Advanced Observability

Goal: Enhanced visibility into system behavior

Steps:

1. Distributed tracing (Cloud Trace integration)
2. Custom dashboards (Cloud Monitoring)
3. SLO-based alerts and tracking
4. Error budget calculations
5. Weekly metrics reports automation
6. Performance profiling and optimization recommendations

Estimated Checkpoints: 8-10

Sprint 6: Feature Enhancements (Future)

Goal: New capabilities based on operational insights

Steps (To be defined):

- Multi-turn conversation optimization
- Session persistence improvements
- Custom model endpoints
- Rate limiting policies
- Advanced caching strategies

Status: Pending user requirements

Next Steps (If Continuing)

Immediate Actions (When Resuming)

1. Review WORK_CHECKPOINT.md this section to understand remaining work
2. Choose next sprint based on priority (Metrics Integration → Production Hardening → Advanced Observability)
3. Check git log for latest commit: `git log --oneline -1`
4. Verify all code is current: `git status`
5. Set up environment: `python -m venv venv && source venv/bin/activate`
6. Install dependencies: `pip install -r requirements.txt`

For Sprint 3 Start

1. Review metrics definitions in `app/utils/metrics_definitions.py`
2. Integrate decorators in `app/services/vertex_agent_client.py`
3. Integrate context managers in `app/services/session_service.py`
4. Update DEPLOYMENT.md with staging environment steps
5. Deploy and validate metrics

Summary

Sprint 2 is **COMPLETE** with all 7 steps implemented, tested, and validated. The IsPilot API now has:

- ☒ Comprehensive OpenAPI documentation
- ☒ Enhanced observability with detailed logging
- ☒ Cloud Monitoring metrics ready for deployment
- ☒ 31/31 validation checkpoints passing
- ☒ Zero regressions from prior phases
- ☒ Production-ready error handling and graceful degradation

All changes committed and ready for staging/production deployment.